

TD Intelligence Artificielle n° 1 (Correction)

Recherche dans un espace d'états

Exercice 1 Traversée du pont

Quatre personnes doivent traverser un pont en moins de 17 minutes. Chacune d'entre elles marche à une vitesse maximale donnée. L'une peut traverser le pont en 1 minute, l'autre en 2 minutes, l'autre en 5 minutes, et la dernière en 10 minutes. Il est impossible de traverser le pont sans torche, et le groupe n'en a qu'une. Enfin, le pont ne peut supporter que le poids de 2 individus.

On cherche à déterminer l'ordre dans lequel les quatre personnes doivent traverser.

1. Choisissez un formalisme permettant de représenter les états possibles.

Correction : Un état peut être représenté par un quintuplet $(P_1, P_2, P_5, P_{10}, T) \in \{\text{gauche}, \text{droite}\}^5$, où P_i est la position de la personne pouvant traverser le pont en i minutes, et T est la position de la torche.

- Espace d'états : $\mathcal{S} = \{\text{gauche}, \text{droite}\}^5$, $|\mathcal{S}| = 2^5$

2. Identifiez, dans ce formalisme, l'état initial et l'ensemble des états finaux.

Correction :

- État initial : $s_{\text{init}} = (\text{gauche}, \text{gauche}, \text{gauche}, \text{gauche}, \text{gauche})$
- États finaux : $\mathcal{F} = \{(P_1, P_2, P_5, P_{10}, T) \in \mathcal{S} \mid P_1 = P_2 = P_5 = P_{10} = \text{droite}\}$

3. Définissez les opérateurs avec leur coût.

Correction :

- Opérateurs :

Nom	Paramètres	Préconditions sur un état $(P_1, P_2, P_5, P_{10}, T) \in \mathcal{S}$	Effet	Coût
traverséeGD	$i, j \in \{1, 2, 5, 10\}$	$P_i = P_j = T = \text{gauche}$	• $P_i \leftarrow \text{droite}$ • $P_j \leftarrow \text{droite}$ • $T \leftarrow \text{droite}$	$\max(i, j)$
traverséeDG	symétrique de traverséeGD (intervertir gauche et droite)			

Remarque : Le cas $i = j$ correspond à la traversée d'une seule personne.

4. Quel algorithme trouvera la solution optimale ?

Correction : Dijkstra ou A* permettrait de trouver une suite d'opérations pour faire traverser le groupe en un temps minimal (on pourrait alors notamment vérifier qu'une solution en moins de 17 minutes, valide selon l'énoncé initial, existe bien). Pour appliquer A* il faut préalablement cependant se doter d'une heuristique minorante (ou admissible), p. ex. le maximum des temps de traversée pour les personnes encore à gauche.

Exercice 2 Problème du taquin

Le taquin est un casse-tête dont le principe est de remettre en ordre des tuiles numérotées. Le jeu se présente sous la forme d'une grille remplie de pièces coulissantes, qui peuvent être déplacées grâce à la présence d'une case vide. Partant d'une situation où les pièces sont mélangées, l'objectif est donc d'atteindre une configuration but dans laquelle chaque tuile est à sa place.

Ici, on considère une version à huit pièces du jeu, en cherchant à réaliser la transformation illustrée par la figure 1.

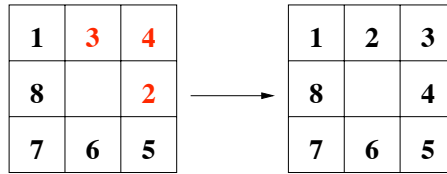


FIGURE 1 – Une situation initiale (à gauche) et la configuration but (à droite) pour la version à huit pièces du taquin.

1. Choisissez un formalisme permettant de représenter les états possibles.

Correction : Un état peut être représenté par une matrice 3×3 , contenant les entiers de 0 à 8 (0 représente la case vide).

- Espace d'états : $\mathcal{S} = \{M \in M_{3,3}(\llbracket 0; 8 \rrbracket) \mid \forall i, j, i', j' \text{ t.q. } (i, j) \neq (i', j'), M_{i,j} \neq M_{i',j'}\}$

2. Combien d'états possibles y a-t-il ?

Correction : $|\mathcal{S}| = 9!$ (nombre de permutations)

3. Identifiez, dans ce formalisme, l'état initial et l'ensemble des états finaux.

Correction :

- État initial : $s_{\text{init}} = \begin{bmatrix} 1 & 3 & 4 \\ 8 & 0 & 2 \\ 7 & 6 & 5 \end{bmatrix}$
- États finaux : $\mathcal{F} = \left\{ \begin{bmatrix} 1 & 2 & 3 \\ 8 & 0 & 4 \\ 7 & 6 & 5 \end{bmatrix} \right\}$

4. Proposez trois heuristiques pour résoudre le jeu de taquin avec l'algorithme A^* .

Correction : Pour pouvoir appliquer A^* (et non seulement A), il faut se munir d'une heuristique dite minorante (ou admissible), c'est-à-dire une heuristique h qui vérifie :

$$\forall s \in \mathcal{S}, h(s) \leq h^*(s)$$

où $h^*(s)$ désigne le coût du plus court chemin depuis l'état s jusqu'à l'état final le plus proche.

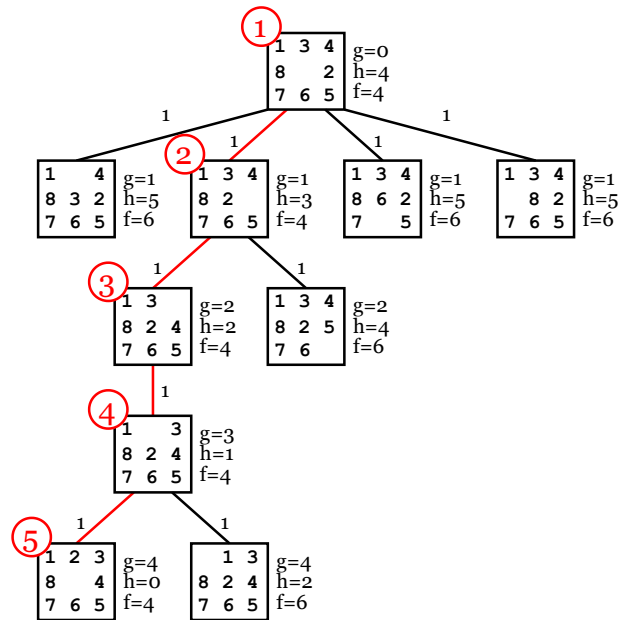
- $h_0 = 0$
 h_0 est une heuristique trivialement minorante (puisque l'on ne considère que des coûts positifs), mais pas informative. Elle ramène A^* à l'algorithme de Dijkstra : $f = g + h = g$.
- $h_1 =$ nombre de pièces mal placées
 h_1 est bien minorante, parce que ramener une pièce à sa place nécessitera *au moins* un déplacement.
- $h_2 =$ somme des distances de Manhattan entre chaque tuile et sa destination

Remarque : La distance de Manhattan est la distance associée à la norme 1. Si A et B sont deux points de coordonnées respectives (i_A, j_A) et (i_B, j_B) , alors : $d(A, B) = |i_A - i_B| + |j_A - j_B|$.

h_2 est bien minorante, parce que ramener une pièce à sa place nécessitera *au moins* de couvrir l'écart vertical et horizontal entre sa position initiale et sa destination.

5. Appliquez l'algorithme A^* avec la meilleure de ces heuristiques.

Correction : De façon évidente, h_2 domine h_1 , qui domine h_0 : $\forall s \in \mathcal{S}, h_0(s) \leq h_1(s) \leq h_2(s) \leq h^*(s)$. La meilleure heuristique est donc h_2 .



Exercice 3 Touriste en Roumanie

On considère un touriste en fin de vacances en Roumanie, qui cherche à rejoindre l'aéroport situé à Bucharest par le plus court chemin. On considère la carte (simplifiée) de la Roumanie de la figure 2 et on dispose par ailleurs d'informations supplémentaires résumées dans le tableau 1.

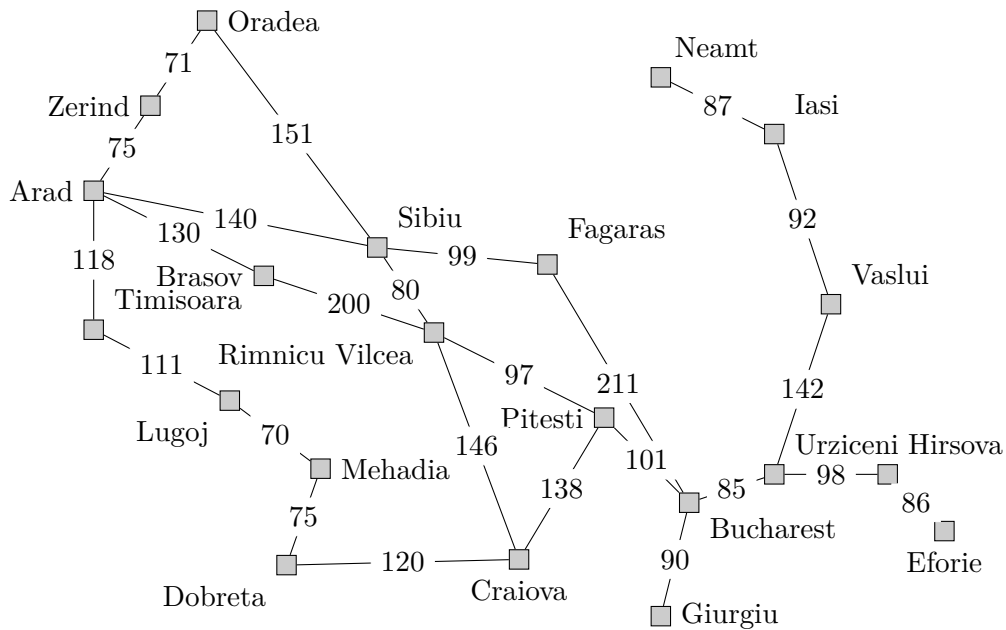
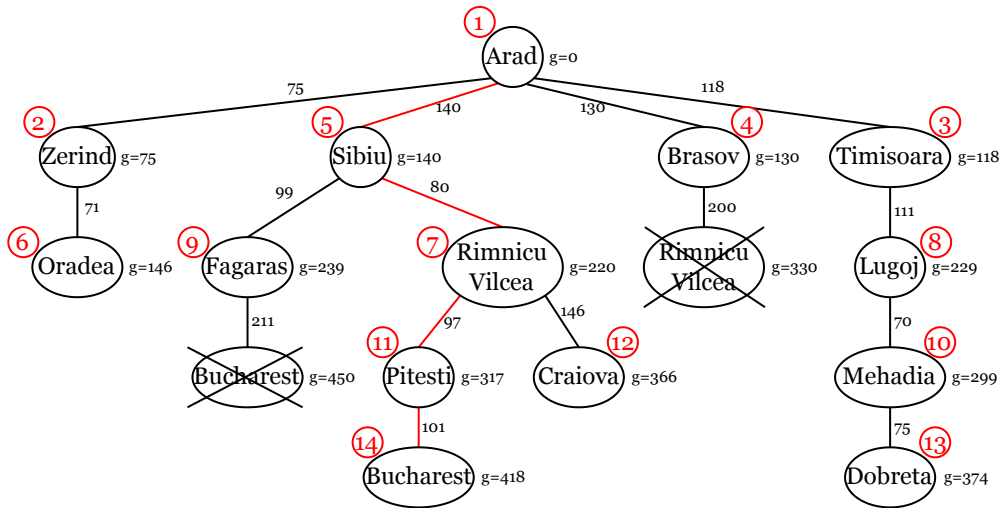


FIGURE 2 – Carte routière simplifiée de la Roumanie, avec longueur des routes (km).

Pour les questions de recherche d'itinéraire ci-dessous, vous dessinerez l'arbre de recherche en indiquant le numéro de l'étape lorsqu'un nœud est choisi dans la frontière. Dans le cas où le chemin menant à un état déjà rencontré serait remis en cause, dessinez un nouveau nœud et barrez l'ancien.

1. Appliquez l'algorithme de Dijkstra pour trouver un itinéraire allant d'Arad à Bucharest.

Correction :



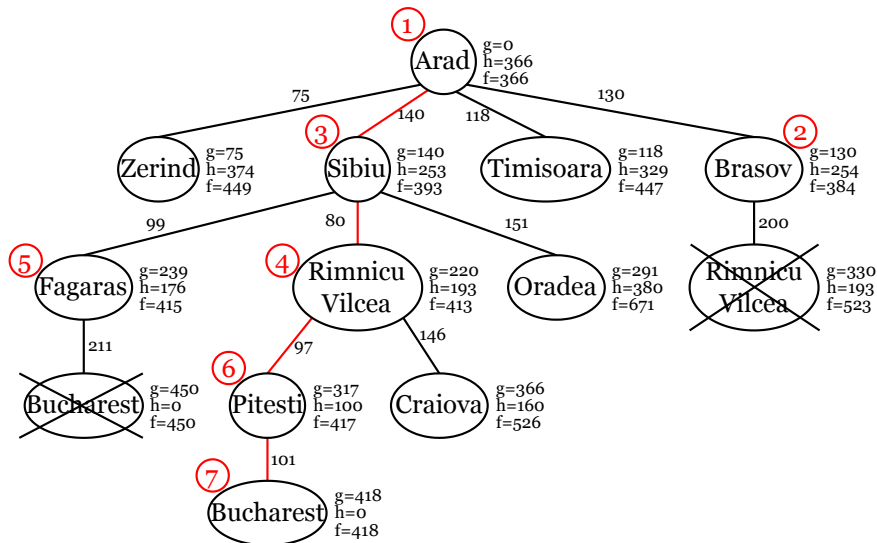
Chemin optimal (418 km) : Arad $\rightarrow$ Sibiu $\rightarrow$ Rimnicu Vilcea $\rightarrow$ Pitesti $\rightarrow$ Bucharest.

- Proposez une heuristique h pour le problème du touriste (autre que $h_0 = 0$). Est-elle minorante (c.-à-d. admissible) ? Justifiez en une phrase.

Correction : La fonction heuristique h est en fait donnée dans le tableau 1, il s'agit de la distance à vol d'oiseau entre la ville considérée et la ville destination, Bucharest. Cette heuristique est bien sûr minorante, puisque la longueur du plus court chemin par route (h^*) ne peut être inférieure à la distance en ligne droite.

- Appliquez l'algorithme A^* pour trouver un itinéraire allant d'Arad à Bucharest.

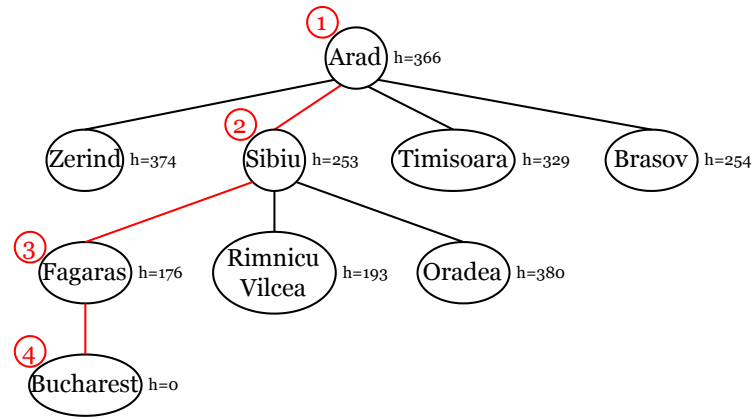
Correction :



Chemin optimal (418 km) : Arad $\rightarrow$ Sibiu $\rightarrow$ Rimnicu Vilcea $\rightarrow$ Pitesti $\rightarrow$ Bucharest.

- Appliquez l'algorithme de recherche gloutonne (c.-à-d. gourmande) pour aller d'Arad à Bucharest.

Correction :



Chemin solution (450 km) : Arad $\rightarrow$ Sibiu $\rightarrow$ Fagaras $\rightarrow$ Bucharest.

Annexe

Arad	366	Fagaras	176	Mehadia	241	Sibiu	253
Brasov	254	Giurgiu	77	Neamt	234	Timisoara	329
Bucharest	0	Hirsova	151	Oradea	380	Urzicen	80
Craiova	160	Iasi	226	Pitesti	100	Vaslui	199
Dobreta	242	Lugoj	244	Rimnicu Vilcea	193	Zerind	374
Eforie	161						

TABLE 1 – Distance à vol d'oiseau jusqu'à Bucharest (km).

Annexe

Algorithme 1 Dijkstra

Entrée :

s_{init} : l'état initial du problème

Sortie :

- une solution, c.-à-d. un chemin entre s_{init} et un état final
ou

- ECHEC, s'il n'existe pas de solution

fonction rechDijkstra(s_{init})

$DejaDev \leftarrow \emptyset$

$Frontiere \leftarrow \{s_{\text{init}}\}$ /* en général on implémente une file de priorité */

$g[s_{\text{init}}] \leftarrow 0$

tant que $Frontiere \neq \emptyset$ **faire**

$s \leftarrow \text{choixGMin}(Frontiere)$

si estFinal(s) **alors**

retourner construireSolution($s, pere$)

sinon

$Frontiere \leftarrow \text{supprimer}(s, Frontiere)$

$DejaDev \leftarrow \text{ajouter}(s, DejaDev)$

pour tout $s' \in \text{successeurs}(s)$ **faire**

si $s' \notin DejaDev$ **et** ($s' \notin Frontiere$ **ou** $g[s] + \text{cout}(s, s') < g[s']$) **alors**

/* soit s' n'a jamais été rencontré avant, soit s' est dans la frontière et le nouveau chemin qui y mène coûte moins que l'ancien */

$pere[s'] \leftarrow s$

$g[s'] \leftarrow g[s] + \text{cout}(s, s')$

$\text{ajouter}(s', Frontiere)$

fin si

fin pour

fin si

fin tant que

/* pas de solution */

retourner ECHEC

fin fonction

Remarques :

- Les chemins menant aux états dans $DejaDev$ ne sont jamais remis en cause (c.-à-d. ils sont optimaux).
- Si tous les arcs ont un coût identique (p.ex. 1), l'algorithme de Dijkstra est équivalent à la recherche en largeur.

Algorithme 2 A/A***Entrée :**

- s_{init} : l'état initial du problème
- h : une fonction heuristique (si h est minorante, l'algorithme devient A*)

Sortie :

- une solution, c.-à-d. un chemin entre s_{init} et un état final
ou
- ECHEC, s'il n'existe pas de solution

fonction rechA(s_{init}, h) $DejaDev \leftarrow \emptyset$ $Frontiere \leftarrow \{s_{\text{init}}\}$ /* en général on implémente une file de priorité */ $g[s_{\text{init}}] \leftarrow 0$ $f[s_{\text{init}}] \leftarrow h(s_{\text{init}})$ **tant que** $Frontiere \neq \emptyset$ **faire** $s \leftarrow \text{choixFMin}(Frontiere)$ **si** estFinal(s) **alors****retourner** construireSolution($s, pere$)**sinon** $Frontiere \leftarrow \text{supprimer}(s, Frontiere)$ $DejaDev \leftarrow \text{ajouter}(s, DejaDev)$ **pour tout** $s' \in \text{successeurs}(s)$ **faire****si** $s' \notin DejaDev \cup Frontiere$ **ou** $g[s] + \text{cout}(s, s') < g[s']$ **alors**/* soit s' n'a jamais été rencontré avant, soit le nouveau chemin qui mène à s'
coûte moins que l'ancien */ $pere[s'] \leftarrow s$ $g[s'] \leftarrow g[s] + \text{cout}(s, s')$ $f[s'] \leftarrow g[s'] + h(s')$ **si** $s' \in DejaDev$ **alors** $\text{supprimer}(s', DejaDev)$ **fin si** $\text{ajouter}(s', Frontiere)$ **fin si****fin pour****fin si****fin tant que**

/* pas de solution */

retourner ECHEC**fin fonction**

Remarques :

- Au contraire de ce qui se passe dans l'algorithme de Dijkstra, les chemins menant aux états dans $DejaDev$ peuvent être remis en cause (c.-à-d. ils ne sont pas nécessairement optimaux, sauf le chemin solution renvoyé à la fin).
- $h = 0$ est une heuristique trivialement minorante (on ne considère que des coûts positifs), mais pas informative. Elle ramène A* à l'algorithme de Dijkstra : $f = g + h = g$.